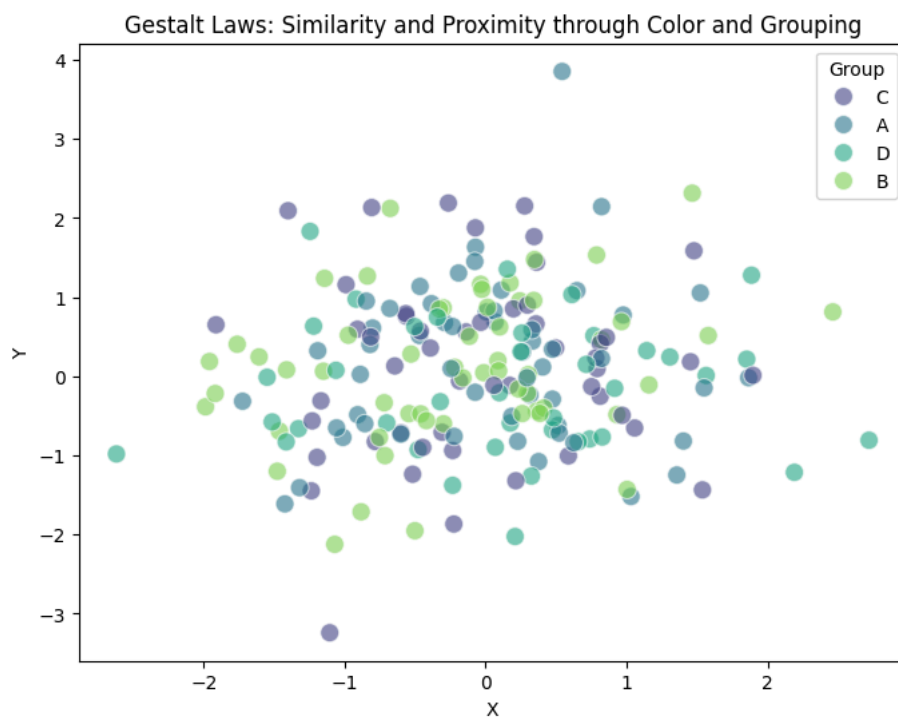


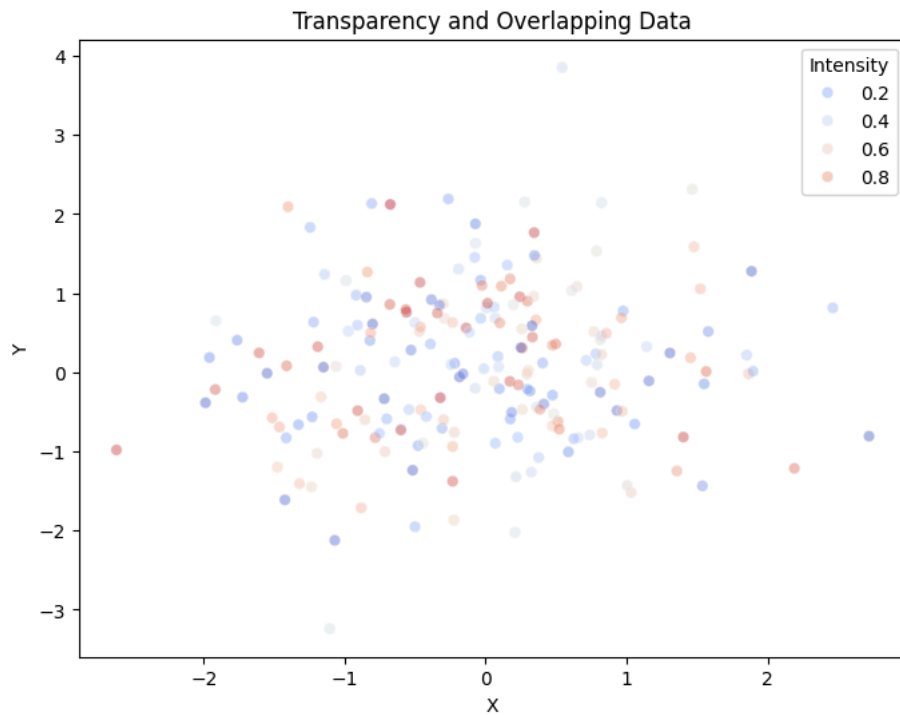
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from matplotlib.animation import FuncAnimation
import plotly.express as px
```

```
np.random.seed(42)
data = pd.DataFrame({
    'X': np.random.randn(200),
    'Y': np.random.randn(200),
    'Group': np.random.choice(['A', 'B', 'C', 'D'], 200),
    'Time': np.random.randint(1, 10, 200),
    'Intensity': np.random.rand(200)
})
```

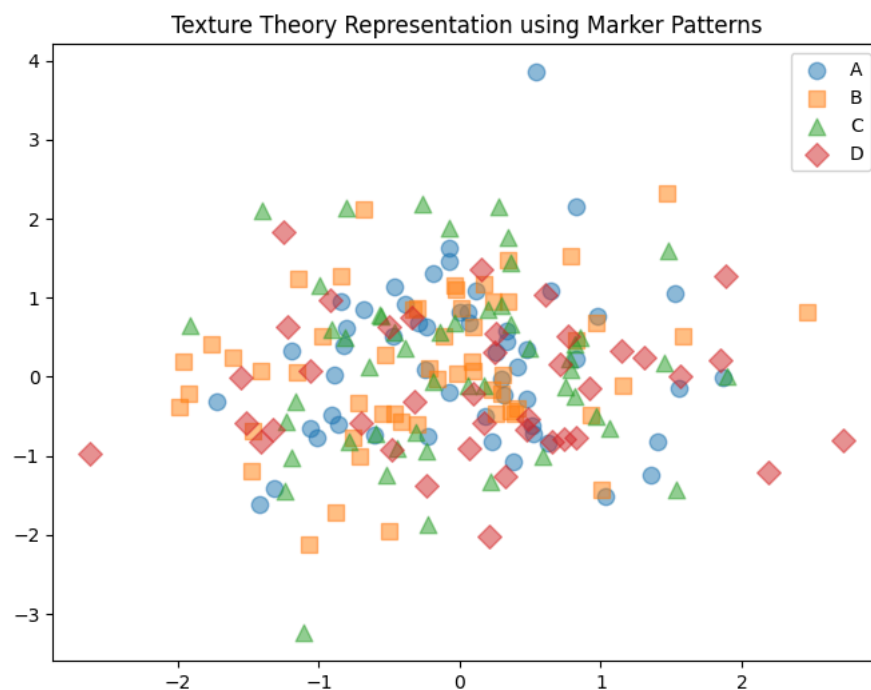
```
plt.figure(figsize=(8,6))
sns.scatterplot(data=data, x='X', y='Y', hue='Group',
                palette='viridis', s=100, alpha=0.6)
plt.title('Gestalt Laws: Similarity and Proximity through Color and Grouping')
plt.show()
```



```
plt.figure(figsize=(8,6))
sns.scatterplot(data=data, x='X', y='Y', hue='Intensity',
                palette='coolwarm', alpha=0.4)
plt.title('Transparency and Overlapping Data')
plt.show()
```

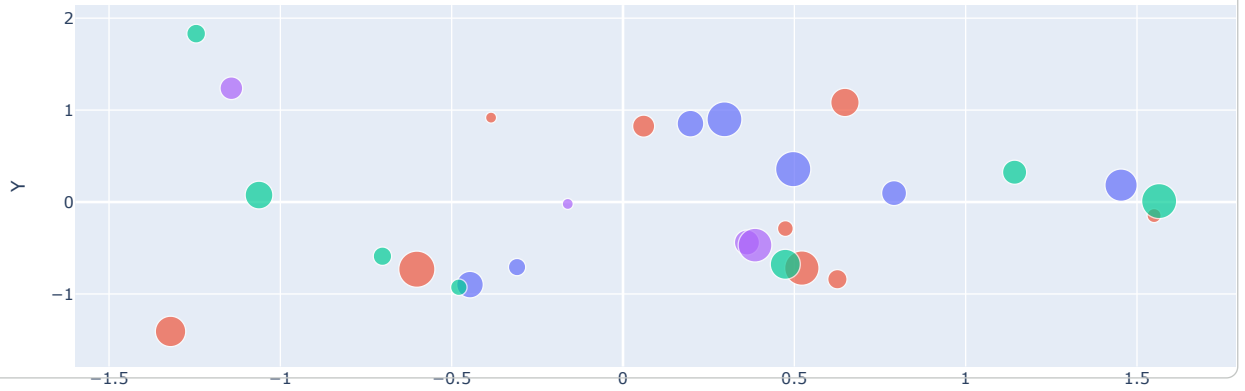


```
plt.figure(figsize=(8,6))
for g, marker in zip(['A','B','C','D'], ['o','s','^','D']):
    subset = data[data['Group']==g]
    plt.scatter(subset['X'], subset['Y'], label=g, alpha=0.5,
                marker=marker, s=80)
plt.title('Texture Theory Representation using Marker Patterns')
plt.legend()
plt.show()
```



```
fig = px.scatter(data, x='X', y='Y', color='Group', size='Intensity',
                 animation_frame='Time', opacity=0.7,
                 title='Data Mapping and Motion Visualization')
fig.show()
```

Data Mapping and Motion Visualization



```
fig, ax = plt.subplots(figsize=(8,6))
sc = ax.scatter([], [], c=[], cmap='plasma', s=100, alpha=0.6)
ax.set_xlim(-3, 3)
ax.set_ylim(-3, 3)
ax.set_title("Patterns in Motion")

def animate(frame):
    subset = data[data['Time'] == frame]
    sc.set_offsets(np.c_[subset['X'], subset['Y']])
    sc.set_array(subset['Intensity'])
    return sc,

ani = FuncAnimation(fig, animate, frames=sorted(data['Time'].unique()),
                    interval=800)

plt.show()
plt.close(fig)
```

